

Neo-Bank Backend Simulator — Final V2 Project Report (Phone-Verified Model)

Author: Dev | **Scope:** End-to-End Fintech Backend System Design without Document KYC

1. Executive Summary

This project implements a production-style backend for a sandbox digital wallet and internal payments platform. Users onboard via OAuth or email/password, must verify their phone number using OTP before accessing the dashboard, and can perform peer-to-peer transfers only with other registered users on the platform. Daily limits are governed by trust tier (phone verification) and subscription plan (Free or Premium). The system uses a ledger-first accounting model, idempotent APIs, sandbox payment gateway integration, dispute workflows, fraud monitoring, admin tooling, chatbot interfaces, strict security hardening, and AWS deployment readiness. Document-based regulatory KYC is intentionally out of scope for V1 to keep the project legally accurate and technically focused.

2. Product Utility

The platform is a closed-loop wallet system: users can add money via a sandbox gateway, hold balances, and send funds only to other registered users inside the platform. External bank rails and national payment networks are future-scope features.

3. Trust & Limit Model

Stage	Phone Verified	Plan	Daily Limit
Signed up only	No	Free	■0
Phone verified	Yes	Free	■2,000
Phone verified	Yes	Premium	■50,000

4. Architecture Overview

The backend is implemented as a Spring Boot modular monolith. Clients call REST APIs secured by JWT authentication and rate limiting. Controllers delegate to domain services that execute inside database transactions. All money movements are recorded in an immutable ledger, and post-commit events are emitted via an outbox table to asynchronous workers for notifications, fraud scanning, and analytics.

5. Internal Modules

- 1 auth
- 2 users
- 3 wallets
- 4 ledger
- 5 transfers

- 6 payments
- 7 subscriptions
- 8 disputes
- 9 fraud
- 10 chatbot
- 11 admin
- 12 audit
- 13 notifications
- 14 ratelimit
- 15 security
- 16 infra

6. Infrastructure Stack

Spring Boot 3, PostgreSQL, Redis, Kafka/RabbitMQ/SQS, Quartz scheduler, Docker, AWS ECS/EC2, RDS, S3, CloudWatch, Secrets Manager, GitHub Actions.

7. Ledger-First Financial Model

Balances are derived from immutable ledger rows. All money movements create paired debit and credit entries; corrections are handled by reversal rows.

8. Subscription Model

Tier	Daily Limit	Benefits
Free	■2k/day	basic transfers
Premium	■50k/day	analytics, APIs, priority queues

9. Database Schema (Core)

users: id, email, phone_number, phone_verified, password_hash, oauth_provider, status, created_at

wallets: id, user_id, currency, status, snapshot_balance

ledger_entries: id, wallet_id, amount, entry_type, transaction_group_id, reference_type, created_at

payments: id, gateway_order_id, gateway_payment_id, status, amount, idempotency_key

subscriptions: id, user_id, plan, status, started_at, expires_at

disputes: id, transaction_group_id, status, reason, opened_at, resolved_at

audit_logs: id, actor_id, action, entity_type, entity_id, timestamp

idempotency_keys: key, request_hash, response, expires_at

chatbot_logs: user_id, intent, action, timestamp

outbox_events: id, type, payload, status

10. Transfer Flow

Authenticate → verify phone status → enforce plan limits → lock wallets → insert debit/credit ledger rows → commit → audit log → outbox event → async workers.

11. Payment/Webhook Flow

Create sandbox order → user pays → webhook received → signature verification → deduplication → ledger credit → wallet update → event emission.

12. Fraud & Monitoring

Velocity checks, abnormal amounts, repeated failures, IP/device throttling, admin freeze workflows, and dispute pipelines protect the system.

13. Ticket/Dispute System

Workflow: OPEN → UNDER_REVIEW → RESOLVED/REJECTED. Opening a dispute freezes funds; resolution posts reversal ledger entries and updates status.

14. Chatbot Engine

Policy-guarded intent engine routes user requests to domain services with confirmations, RBAC, and audit logging.

15. Security Hardening

JWT auth, RBAC, Redis rate limits, DTO validation, encrypted secrets, webhook HMAC verification, OWASP Top-10 defenses, CSRF protection, and audit trails.

16. Observability

Structured logs, metrics dashboards, tracing, queue-depth alerts, and fraud alarms.

17. Deployment Strategy

Local Docker Compose; production ECS/EC2, RDS PostgreSQL, Redis, S3, CI/CD, blue-green deploys.

18. Scaling & Microservice Evolution

Sharding wallets, partitioning ledgers, read replicas, cache layers, isolated fraud workers, and future domain-based service extraction with saga workflows.

19. Execution Roadmap

Phase	Focus
1	Ledger + transfers
2	Payments + subscriptions
3	Disputes + fraud
4	Chatbot + admin tooling
5	Infra + docs

20. Resume Description

Designed a sandbox digital wallet backend with phone-based onboarding, premium-gated limits, ledger-first accounting, sandbox payment gateway integration, dispute workflows, fraud monitoring, chatbot interfaces, OWASP-aligned security, and AWS-ready infrastructure using Spring Boot.